

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)**



MICRO PROJECT REPORT

PCADT503: MACHINE LEARNING

**IMAGE CLASSIFICATION ON THE CIFAR-10 DATASET USING
CONVOLUTIONAL NEURAL NETWORKS**

Submitted by

**ABINAND CV (VML24AD006)
AKARSH PREM (VML24AD014)
ANNLIA JISHA RAJEEV (VML24AD031)
DENZ JOSEPH (VML24AD044)
JISTO JOSE (VML24AD065)
SHRIYA SREEJEESH (VML24AD108)**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)**

**Vimal Jyothi Engineering Collage Chemperi
Academic year 2026-2027**



VIMAL JYOTHI
ENGINEERING COLLEGE (AUTONOMOUS)
Jyothi Nagar, Chemperi - 670632, Kannur D.T, Kerala



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND DATA SCIENCE)

CERTIFICATE

This is to certify that this report titled “**Image Classification on the CIFAR-10 Dataset using Convolutional Neural Networks**” is a bonafide record of the Machine Learning (PCADT503) mini project work carried out by **ABINAND CV (VML24AD006)**, **AKARSH PREM (VML24AD014)**, **ANNLIA JISHA RAJEEV (VML24AD031)**, **DENZ JOSEPH (VML24AD044)**, **JISTO JOSE (VML24AD065)** and **SHRIYA SREEJESH (VML24AD108)**, students of B.Tech Computer Science and Engineering, Semester V, at Vimal Jyothi Engineering College, during the academic year 2026–2027, in partial fulfilment of the requirements for the course **PCADT503 – Machine Learning**. The work presented in this report is the result of the collective investigation, design, implementation, and evaluation carried out by the above-mentioned students under the guidance of the course faculty. The project work has been completed as a group and has not been submitted elsewhere for the award of any degree or diploma.

Ms Aswathy T S
(Faculty Supervisor)
Assistant Professor
Dept of CSE(ADS)
Vimal Jyothi Engineering
Collage, Chemperi

Dr Manoj V Thomas
(Program Coordinator)
Professor
Dept of CSE (ADS)
Vimal Jyothi Engineering
Collage, Chemperi

DECLARATION

We hereby declare that this project report titled “**Image Classification on the CIFAR-10 Dataset using Convolutional Neural Networks**”, submitted for the course **PCADT503 – Machine Learning**, is an original record of the work carried out by us under the guidance of the respective course faculty at Vimal Jyothi Engineering College. We further declare that the model architecture, code implementation, experimental results, and analysis presented in this report were collectively developed and executed by us on the **Google Colaboratory platform**, and that all external concepts, theories, and reference materials used in the preparation of this report have been duly acknowledged in the References section. This report has not been submitted, in part or in full, to any other institution or university for the award of any degree, diploma, or similar title.

Place: Chemperi

Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the **Department of Computer Science and Engineering, Vimal Jyothi Engineering College**, for providing us with the opportunity and resources to undertake this Machine Learning mini project as part of the **PCADT503 – Machine Learning** course curriculum. We are deeply thankful to our **course faculty** for their invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this project. Their insights into convolutional neural network design and evaluation methodology greatly helped shape the direction and successful completion of this work. We also extend our sincere gratitude to the **Head of the Department and the faculty members of the CSE Department** for fostering an environment that encourages practical, hands-on learning in emerging areas such as deep learning and computer vision. We would also like to acknowledge the open-source community behind **TensorFlow, Keras, scikit-learn, Matplotlib, and Seaborn**, whose well-documented libraries made the implementation of this project feasible within the **Google Colaboratory** environment. Finally, we express our heartfelt thanks to our **classmates and peers** for their moral support, encouragement, and useful discussions throughout the course of this project. We are grateful to everyone who contributed directly or indirectly to the successful completion of this work.

ABSTRACT

Image classification is one of the most fundamental and extensively studied problems in computer vision, forming the basis for a wide range of downstream applications such as object detection, autonomous navigation, medical image analysis, and visual surveillance. This project addresses the problem of multi-class image classification using the CIFAR-10 benchmark dataset, which consists of 60,000 colour images of size 32×32 pixels distributed uniformly across ten object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

A Convolutional Neural Network (CNN) was designed and implemented using the TensorFlow and Keras deep learning frameworks to automatically learn discriminative spatial features from raw pixel data. The baseline architecture consists of three convolutional blocks employing 32 and 64 filters respectively, interleaved with max-pooling and dropout regularization layers, followed by a fully connected classification head with a softmax output layer over ten classes. The model was compiled using the Adam optimizer and categorical cross-entropy loss, and trained for 20 epochs on 50,000 training images with an 80:20 internal validation split.

To examine the effect of regularization through synthetic data expansion, a second variant of the same architecture was trained using real-time data augmentation – comprising random rotations, width and height shifts, and horizontal flips – applied via Keras’s ImageDataGenerator. Both models were evaluated on an unseen test set of 10,000 images using accuracy, loss, a full classification report, and a confusion matrix.

The baseline CNN achieved a final test accuracy of **71.06%** with a test loss of **0.8420**, while the augmented CNN achieved a comparable test accuracy of **70.02%** with a test loss of **0.8608**. Detailed per-class analysis revealed that structurally distinctive classes such as *automobile*, *ship*, and *truck* were classified with high precision and recall, whereas visually similar animal classes – particularly *cat* and *dog* – exhibited considerable inter-class confusion. This report documents the complete methodology, implementation, and results of the project, and concludes with a discussion of the observed trade-offs between model generalization and training stability, along with directions for future improvement such as batch normalization, deeper residual architectures, and transfer learning.

TABLE OF CONTENTS

Introduction	9
1.1 Background	9
1.2 Motivation	9
1.3 Problem Statement	10
1.4 Objectives	10
1.5 Scope of the Project	10
Literature Review	11
2.1 Evolution of Convolutional Neural Networks	11
2.2 The CIFAR-10 Benchmark in Prior Work	11
2.3 Regularization Techniques	12
2.4 Optimization Algorithms	12
2.5 Loss Functions for Multi-Class Classification	13
2.6 Summary of Literature Review	13
Dataset Description	14
3.1 Overview of CIFAR-10	14
3.2 Class Labels	14
3.3 Class-wise Distribution	15
3.4 Image Properties	16
3.5 Data Preprocessing Applied	16
Tools, Technologies and System Requirements	17
4.1 Development Environment	17
4.2 Programming Language and Core Libraries	17
4.3 Key Framework APIs Used	17
4.4 Hardware and Computational Considerations	18
4.5 Minimum System Requirements for Reproduction	18
Proposed Methodology and System Design	19
5.1 Overall Methodology	19
5.2 Theoretical Background of CNN Components	20
5.3 Proposed CNN Architecture	21
5.4 Compilation Configuration	22
5.5 Data Augmentation Design	22

Implementation	24
6.1 Implementation Overview	24
6.2 Library Imports and Dataset Loading	24
6.3 Baseline Model Definition	24
6.4 Model Compilation	25
6.5 Label Inspection and One-Hot Encoding	25
6.6 Training the Baseline Model	26
6.7 Visualizing Training Progress	26
6.8 Model Evaluation on the Test Set	27
6.9 Generating Predictions	27
6.10 Confusion Matrix and Classification Report	27
6.11 Qualitative Inspection of Predictions	28
6.12 Data Augmentation Pipeline	28
6.13 Augmented Model Definition and Training	28
6.14 Final Evaluation and Model Comparison	29
Results and Discussion	30
7.2 Baseline Model – Training Behavior	30
7.3 Baseline Model – Final Test Evaluation	32
7.4 Confusion Matrix Analysis	33
7.5 Per-Class Classification Report	34
7.6 Qualitative Prediction Inspection	35
7.7 Augmented Model – Training Behavior	36
7.8 Augmented Model – Final Test Evaluation	37
7.9 Comparative Analysis: Baseline vs. Augmented CNN	37
7.10 Discussion	37
Testing and Validation Strategy	39
Conclusion and Future Scope	41
References	43
Appendix: Complete Source Code	44

LIST OF FIGURES

3.1	CIFAR-10 Class-wise Image Distribution	15
5.1	Project Methodology Workflow	19
5.2	Baseline CNN Architecture Diagram	21
5.3	Examples of Data Augmentation	23
7.1	Baseline CNN – Training vs Validation Accuracy	31
7.2	Baseline CNN – Training vs Validation Loss	31
7.3	Confusion Matrix – Baseline CNN	33
7.4	Per-Class Precision, Recall and F1-Score	34
7.5	Sample Predictions on Test Images	35
7.6	Test Accuracy Comparison – Basic vs Augmented CNN	37

40LIST OF TABLES

2.1	Summary of Key Concepts and Relevance	13
3.1	CIFAR-10 Class Labels	14
3.2	CIFAR-10 Dataset Specifications	16
4.1	Software Stack Used in the Project	17
4.2	Recommended Reproduction Environment	18
5.1	Layer Configuration and Parameter Count	22
5.2	Model Compilation Hyperparameters	22
5.3	Data Augmentation Parameters	22
7.1	Baseline CNN Epoch-wise Training Log	30
7.2	Baseline CNN Final Test Evaluation	32
7.3	Per-Class Precision, Recall, F1-Score	34
7.4	Augmented CNN Epoch-wise Training Log	36
7.5	Augmented CNN Final Test Evaluation	37
8.1	Evaluation Metrics Used and Their Purpose	40

1. INTRODUCTION

1.1 Background

The ability to automatically interpret visual information is one of the defining capabilities of modern artificial intelligence systems. Among the many tasks that fall under computer vision, **image classification** – the task of assigning a single semantic label to an entire image from a fixed set of categories – remains a foundational problem, both because of its direct practical utility and because it serves as a stepping stone toward more complex vision tasks such as object detection, semantic segmentation, and image captioning. For decades, image classification systems relied on hand-crafted feature extraction techniques such as edge detectors, color histograms, and texture descriptors (e.g., SIFT, HOG), which were then fed into conventional classifiers such as Support Vector Machines or k-Nearest Neighbors. While effective for constrained problems, these pipelines struggled to generalize across the enormous variability present in natural images – differences in lighting, pose, occlusion, and background clutter. The introduction of **Convolutional Neural Networks (CNNs)**, and in particular the success of deep architectures on large-scale benchmarks from 2012 onward, fundamentally changed this landscape by allowing models to learn hierarchical visual features directly from raw pixel data, without manual feature engineering. This project explores the design, training, and evaluation of a CNN-based image classifier on the **CIFAR-10** dataset, a widely used benchmark in the deep learning community for prototyping and comparing image classification architectures. Despite the relatively small size of its images (32×32 pixels), CIFAR-10 remains a non-trivial classification problem due to low resolution, intra-class variability, and visually overlapping categories such as cats, dogs, and other animals.

1.2 Motivation

The motivation behind this project is threefold:

- **Conceptual understanding:** To gain hands-on experience with the core building blocks of a CNN – convolutional layers, pooling operations, activation functions, and regularization techniques – and to observe how each contributes to the model’s ability to learn visual patterns.
- **Practical evaluation methodology:** To move beyond a single accuracy figure and adopt a rigorous evaluation methodology using confusion matrices, per-class precision/recall/F1-scores, and qualitative inspection of correct and incorrect predictions.
- **Regularization and generalization:** To empirically investigate whether real-time data augmentation improves a model’s ability to generalize to unseen data

1.3 Problem Statement

Given a set of 60,000 labelled 32×32 color images belonging to 10 mutually exclusive object categories, design and train a convolutional neural network capable of accurately predicting the correct category for previously unseen images, and rigorously evaluate its classification performance both quantitatively and qualitatively

The problem is formulated as a supervised, single-label, multi-class classification task. The input space consists of RGB images of dimension $32 \times 32 \times 3$, and the output space consists of a probability distribution over 10 discrete class labels, obtained through a SoftMax activation at the final layer of the network.

1.4 Objectives

The specific objectives set out for this project are as follows: To load, preprocess, and normalize the CIFAR-10 dataset for use with a Kera's Sequential CNN model.

1. To design a baseline CNN architecture comprising convolutional, pooling, dropout, and dense layers suitable for 10-class image classification.
2. To train the baseline model for 20 epochs and analyses its learning behavior using training and validation accuracy/loss curves.
3. To evaluate the trained model on an independent test set using accuracy, loss, a full classification report, and a confusion matrix.
4. To implement a real-time data augmentation pipeline (rotation, translation, and horizontal flipping) and train a second model of identical architecture using the augmented data stream.
5. To comparatively analyses the performance of the baseline and augmented models, identifying the classes and error patterns that most affect overall accuracy.

1.5 Scope of the Project

The scope of this project is limited to supervised image classification on the CIFAR-10 dataset using a custom-built, moderately deep CNN trained from scratch – that is, without using pretrained weights or transfer learning. The project covers the complete pipeline from raw data loading through to final model evaluation, but does not extend to deployment, real-time inference. Model architectures more advanced than the baseline CNN – such as residual networks, attention-based vision transformers, or ensemble methods – are discussed only in the context of future work.

2. LITERATURE REVIEW

2.1 Evolution of Convolutional Neural Networks

The evolution of modern Convolutional Neural Networks (CNNs) can be traced back to LeCun’s LeNet-5, one of the earliest successful convolutional models developed for handwritten digit recognition. LeNet-5 demonstrated that combining convolutional and pooling layers could automatically learn hierarchical spatial features from images, establishing the foundation for later CNN architectures. The deep learning era in image classification was further advanced by Alex Net, which achieved a major improvement over traditional hand-crafted feature methods in the ImageNet Large Scale Visual Recognition Challenge. Alex Net popularized important techniques such as the REL activation function, dropout regularization, and GPU-based training, making deep neural networks more efficient and practical. Later architectures introduced further improvements: Vignette demonstrated the effectiveness of stacking small 3×3 convolutional filters, Google Net introduced Inception modules to capture features at multiple scales using parallel filters, and Resnet employed residual or skip connections to overcome the vanishing gradient problem and enable the training of very deep networks. These architectural developments have significantly influenced modern CNN design and provide the foundation for the architectural choices adopted in this project.

2.2 The CIFAR-10 Benchmark in Prior Work

CIFAR-10 has served as a standard testbed for evaluating new CNN architectures and training techniques precisely because its small image size allows for rapid experimentation while still presenting a genuinely challenging classification problem due to low resolution and inter-class visual similarity. Early hand-crafted-feature approaches on CIFAR-10 achieved test accuracies in the range of 60–65%, while deep CNN-based approaches with modern regularization and data augmentation techniques have been shown in the literature to exceed 90–95% test accuracy when sufficiently deep architectures, batch normalization, and extensive augmentation pipelines are employed. The moderate-depth baseline architecture implemented in this project, which does not use batch normalization or residual connections, is expected to plateau in the 65–75% accuracy range – consistent with the results obtained.

2.3 Regularization Techniques

Dropout

Dropout is a regularization technique in which a random subset of neuron activations is set to zero during each training iteration, with the dropped units resampled on every forward pass. This prevents co-adaptation of neurons and forces the network to learn redundant, distributed representations, which has been shown to substantially reduce overfitting in deep networks with a large number of parameters. In the CNN architecture implemented in this project, dropout is applied after each pooling stage (rate 0.25) and before the final classification layer (rate 0.5), following common practice of using a higher dropout rate closer to the output layer where the risk of overfitting on high-level features is greatest

Data Augmentation

Data augmentation artificially expands the effective size and diversity of a training set by applying label-preserving transformations to existing images – such as rotation, translation, flipping, zooming, or color jittering – at training time. Because the transformations are applied stochastically and on-the-fly, the network effectively never sees the exact same image twice, which can improve its ability to generalize to variations present in unseen data. However, the literature also notes that overly aggressive augmentation, or augmentation that does not match the natural variability of the target domain, can sometimes slow convergence or slightly reduce peak accuracy within a fixed training budget

Batch Normalization

Although not used in the baseline architecture implemented here, batch normalization is widely discussed in the literature as a technique that normalizes layer activations across a mini-batch, which stabilizes and accelerates training by reducing internal covariate shift. Its omission from the present architecture is noted as a candidate direction for future improvement.

Optimization Algorithms

Training a CNN requires iteratively adjusting millions of weight parameters to minimize a loss function via gradient descent. While vanilla stochastic gradient descent (SGD) remains a strong baseline optimizer, adaptive optimization algorithms such as **Adam** (Adaptive Moment Estimation) combine the benefits of momentum-based updates with per-parameter adaptive learning rates, computed from running estimates of the first and second moments of the gradients. Adam has become a default choice in much of the deep learning literature due to its fast convergence and relative insensitivity to hyperparameter choice, which motivated its selection as the optimizer for both models trained in this project.

Loss Functions for Multi-Class Classification

For multi-class, single-label classification problems, cross-entropy loss is the standard choice, as it directly measures the divergence between the predicted probability distribution (produced by a SoftMax output layer) and the true one-hot encoded label distribution. Keras provides two related implementations: `categorical_crossentropy`, which expects one-hot encoded integer labels, and `sparse_categorical_crossentropy`, which expects integer class indices directly. Both are mathematically equivalent and differ only in the expected label encoding format, a distinction that is discussed further in the implementation notes of Section 6.4.

Summary of Literature Review

Concept	Origin / Context	Relevance to This Project
Convolution & Pooling	LeNet-5	Core feature-extraction layers of the baseline CNN
ReLU Activation, Dropout, GPU Training	AlexNet	ReLU and dropout directly used in the model
Small Stacked Filters (3×3)	VGGNet	All convolutional layers use 3×3 kernels
Residual Connections	ResNet	Identified as future work (Chapter 9)
Adam Optimiser	Kingma & Ba	Used to compile and train both models
Data Augmentation	General DL Practice	Implemented via Keras ImageDataGenerator
Batch Normalisation	Ioffe & Szegedy	Not used; proposed as future enhancement

Table 2.1: Summary of Key Concepts and Their Relevance to This Project

3.DATASET DESCRIPTION

3.1 Overview of CIFAR-10

The CIFAR-10 dataset is a widely used benchmark dataset for image classification, consisting of 60,000 color images of size 32×32 pixels, evenly distributed across 10 mutually exclusive classes with 6,000 images per class. The dataset is pre-split into 50,000 training images and 10,000 test images, and is directly accessible through the Keras's datasets API, which was the loading mechanism used in this project via `tensorflow.keras.datasets.cifar10.load_data()`.

3.2 Class Labels

The ten classes represented in CIFAR-10, along with their corresponding integer label indices as returned by the dataset loader, are listed in Table 3.1.

Label Index	Class Name	Category Type
0	airplane	Vehicle
1	automobile	Vehicle
2	bird	Animal
3	cat	Animal
4	deer	Animal
5	dog	Animal
6	frog	Animal
7	horse	Animal
8	ship	Vehicle
9	truck	Vehicle

Table 3.1: CIFAR-10 Class Labels

3.3 Class-wise Distribution

A key property of CIFAR-10 is that it is a perfectly **balanced** dataset – every class contributes exactly 5,000 images to the training set and 1,000 images to the test set, as shown in Figure 3.1. This balance is important from a methodological standpoint, as it means the overall accuracy metric is not skewed by class imbalance, and per-class metrics such as precision and recall can be compared directly across classes without needing to normalize for support.

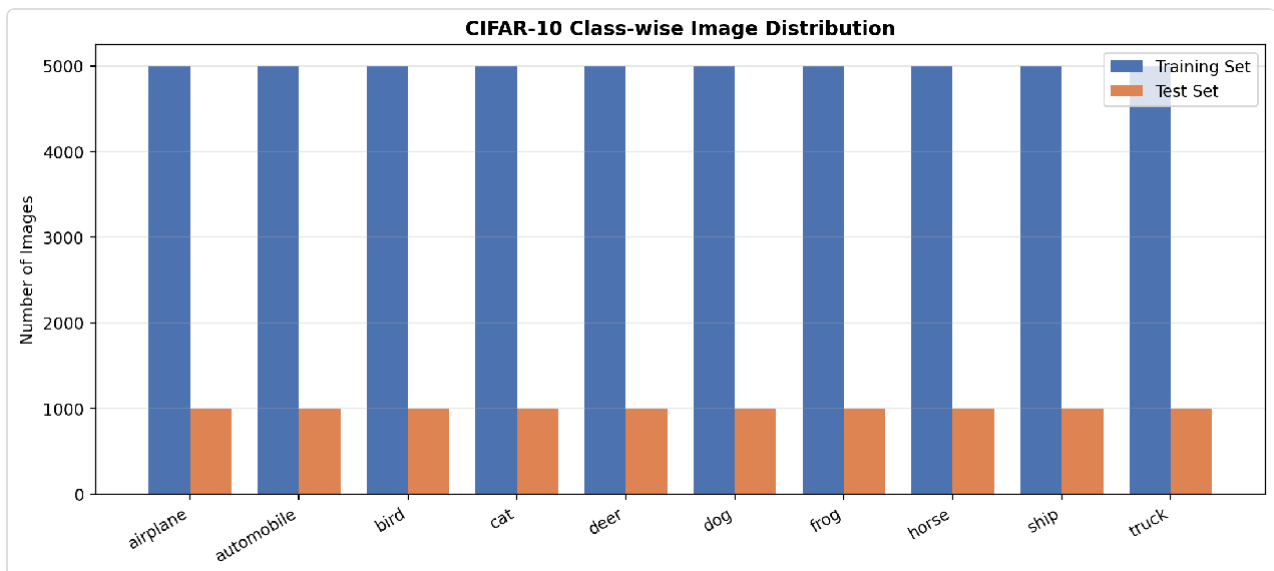


Figure 3.1: CIFAR-10 class-wise image distribution across the training and test sets. Each of the 10 classes contributes exactly 5,000 training images and 1,000 test images.

3.4 Image Properties

Property	Value
Image dimensions	32 × 32 pixels
Colour channels	3 (RGB)
Total images	60,000
Training images	50,000
Test images	10,000
Number of classes	10
Images per class	6,000 (5,000 train + 1,000 test)
Original pixel value range	0 – 255 (uint8)
Raw label array shape	y_train: (50000, 1), y_test: (10000, 1)

Table 3.2: CIFAR-10 Dataset Specifications

3.5 Data Preprocessing Applied

Two preprocessing steps were applied to the raw CIFAR-10 data before it was used for model training, as observed directly in the project implementation:

Pixel Normalization

The raw pixel intensity values, originally stored as 8-bit unsigned integers in the range [0, 255], were cast to 32-bit floating point and rescaled to the range [0, 1] by dividing by 255.0:

```
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

Normalizing pixel values to a small, consistent numerical range is standard practice in deep learning, as it helps gradient-based optimizers converge more smoothly and prevents any single input feature from disproportionately dominating the initial weight updates.

Label Encoding

The raw class labels were provided as integer indices in a column vector of shape (N, 1). Since the model was compiled with `categorical_crossentropy` loss (which expects one-hot encoded targets rather than sparse integer labels), the labels were converted using Keras's `to_categorical()` utility:

```
from tensorflow.keras.utils import to_categorical
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

For example, the integer label 6 (frog) is transformed into the 10-dimensional one-hot vector [0,0,0,0,0,1,0,0,0], where the index of the single '1' corresponds to the class index in Table 3.1.

4.TOOLS, TECHNOLOGIES AND SYSTEM REQUIREMENTS

4.1 Development Environment

The entire project was developed and executed on **Google Collaboratory (Collab)**, a free, cloud-hosted Jupyter notebook environment that provides on-demand access to computational resources without requiring local installation of deep learning frameworks. Colab was chosen for its zero-configuration setup, built-in support for TensorFlow/Keras, and ability to persist notebooks directly to Google Drive, which is reflected in the notebook source URL referenced throughout this project's development.

4.2 Programming Language and Core Libraries

Component	Purpose in This Project
Python 3	Primary programming language for the entire pipeline
TensorFlow / Keras	Deep learning framework used to build, compile, and train the CNN models (<code>layers</code> , <code>models.Sequential</code>)
NumPy	Array manipulation, e.g. flattening labels and computing <code>argmax</code> over predicted probabilities
Matplotlib	Plotting training/validation accuracy and loss curves, sample image grids, and comparison bar charts
Seaborn	Rendering the annotated confusion matrix heatmap
scikit-learn	<code>confusion_matrix</code> , <code>classification_report</code> , and <code>train_test_split</code> utilities for evaluation and stratified splitting
Keras <code>ImageDataGenerator</code>	Real-time data augmentation (rotation, shifting, flipping) during training of the augmented model

Table 4.1: Software Stack Used in the Project

4.3 Key Framework APIs Used

- `tensorflow.keras.datasets.cifar10` – direct loading of the CIFAR-10 dataset as NumPy arrays.
- `tensorflow.keras.models.Sequential` – linear stack-of-layers API used to define both the baseline and augmented CNN architectures.
- `tensorflow.keras.layers` – Input, Conv2D, MaxPooling2D, Dropout, Flatten, and Dense layers.
- `tensorflow.keras.utils.to_categorical` – one-hot encoding of integer class labels.
- `tensorflow.keras.preprocessing.image.ImageDataGenerator`
- `sklearn.model_selection.train_test_split` – stratified train/validation split for the augmented model.
- `sklearn.metrics.confusion_matrix`, `classification_report` – quantitative evaluation utilities.

4.4 Hardware and Computational Considerations

Model training was performed within the compute resources allocated by the Google Colab runtime. Each training epoch for the baseline model took approximately 51–85 seconds, and each epoch of the augmented model – which additionally performs real-time image transformations on every batch – took approximately 73–85 seconds, reflecting the modest additional computational overhead introduced by the augmentation pipeline. Over 20 epochs, total training time for each model was in the order of 25–28 minutes.

4.5 Minimum System Requirements for Reproduction

Requirement	Recommendation
Python version	3.9 or higher
TensorFlow version	2.x (with Keras integrated)
RAM	At least 8 GB (12-16 GB recommended for smooth in-memory handling of CIFAR-10 arrays)
GPU	Optional but recommended for faster training (Colab's free-tier GPU runtime is sufficient)
Storage	Under 500 MB for dataset caching and model checkpoints
Internet connection	Required for the initial CIFAR-10 dataset download via the Keras API

Table 4.2: Recommended Environment for Reproducing This Project

5.PROPOSED METHODOLOGY AND SYSTEM DESIGN

5.1 Overall Methodology

The methodology adopted for this project follows a standard supervised deep learning pipeline, illustrated in Figure 5.1: the CIFAR-10 dataset is loaded and preprocessed, split into training, validation, and test partitions, optionally passed through a data augmentation pipeline, and used to train a convolutional neural network whose performance is subsequently evaluated using a battery of quantitative metrics.



Figure 5.1: End-to-end methodology workflow followed in this project, from raw dataset to final performance analysis.

5.2 Theoretical Background of CNN Components

A Convolutional Neural Network is a class of deep neural network specifically designed to process grid-structured data such as images. Unlike a fully connected network, a CNN exploits the spatial locality of pixel data through **local receptive fields** and **weight sharing**, dramatically reducing the number of trainable parameters compared to a fully connected architecture of comparable depth. The key building blocks used in the architecture designed for this project are described below.

Convolutional Layer (Conv2D)

A convolutional layer applies a set of learnable filters (kernels) across the spatial dimensions of the input, computing a dot product between the filter weights and a local patch of the input at every spatial position. Each filter produces a two-dimensional **feature map** that highlights the presence of a particular visual pattern – such as an edge, corner, or texture – at each location in the image. Stacking multiple filters allows a single convolutional layer to detect many different patterns simultaneously; stacking multiple convolutional layers allows the network to progressively compose low-level features (edges) into higher-level, more abstract features (shapes, object parts). In this project, all convolutional layers use 3×3 kernels with ReLU activation, and no padding ('valid' convolution), which is why the spatial dimensions shrink slightly after each convolution (e.g., $32 \times 32 \rightarrow 30 \times 30$ after the first Conv2D layer).

Activation Function – ReLU

The Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, is applied after every convolutional and dense layer (except the output layer) in the network. ReLU introduces the non-linearity necessary for the network to model complex functions, while being computationally inexpensive and avoiding the vanishing-gradient issues associated with saturating activation functions such as sigmoid or tanh.

Pooling Layer (MaxPooling2D)

Max-pooling layers perform spatial downsampling by sliding a window (2×2 in this project) across the feature map and retaining only the maximum activation value within each window. This reduces the spatial resolution of the feature maps (e.g., $30 \times 30 \rightarrow 15 \times 15$), which decreases the computational cost of subsequent layers, provides a degree of translation invariance, and helps control overfitting by discarding fine-grained spatial information that is not essential for classification.

Dropout Regularisation

As discussed in Section 2.3.1, dropout randomly deactivates a fraction of neurons during each training step. The architecture designed for this project applies dropout with rate 0.25 after each of the first two pooling stages, and a higher rate of 0.5 immediately before the final output layer, following the common heuristic of increasing dropout strength closer to the most overfitting-prone, high-capacity layers of the network.

Flatten and Fully Connected (Dense) Layers

After the final convolutional block, the resulting 3-dimensional feature volume ($4 \times 4 \times 64 = 1,024$ values) is flattened into a single 1-dimensional feature vector. This vector is passed through a fully connected (Dense) layer of 64 units with ReLU activation, which combines the spatially distributed features extracted by the convolutional layers into a compact representation suitable for final classification.

Output Layer – Softmax

The final Dense layer contains 10 units – one per CIFAR-10 class – with a softmax activation function, which converts the raw output scores (logits) into a normalised probability distribution over the 10 classes, such that all outputs are non-negative and sum to 1. The predicted class is taken as the index of the highest probability output (via `np.argmax`), as implemented in Section 6.6.

5.3 Proposed CNN Architecture

Figure 5.2 illustrates the complete layer-by-layer architecture of the baseline CNN model designed for this project, showing the transformation of tensor shape at each stage from the raw $32 \times 32 \times 3$ input image to the final 10-class probability output.

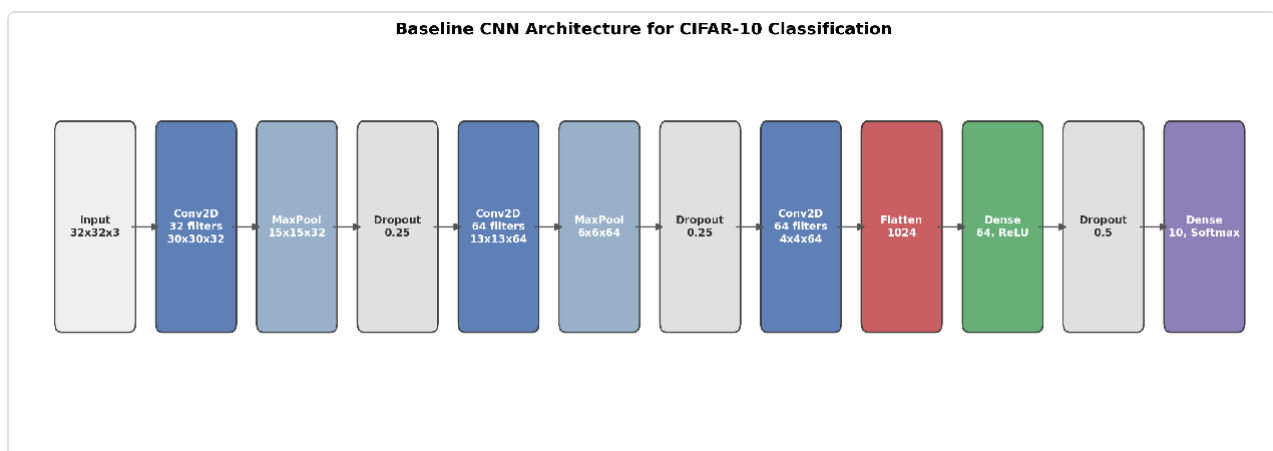


Figure 5.2: Layer-by-layer architecture of the baseline CNN, showing filter counts, output tensor shapes, and regularisation placement.

N o	Layer (Type)	Output Shape	Parameters
1	Conv2D (32 filters, 3×3, ReLU)	(None, 30, 30, 32)	896
2	MaxPooling2D (2×2)	(None, 15, 15, 32)	0
3	Dropout (0.25)	(None, 15, 15, 32)	0
4	Conv2D (64 filters, 3×3, ReLU)	(None, 13, 13, 64)	18,496
5	MaxPooling2D (2×2)	(None, 6, 6, 64)	0
6	Dropout (0.25)	(None, 6, 6, 64)	0
7	Conv2D (64 filters, 3×3, ReLU)	(None, 4, 4, 64)	36,928
8	Flatten	(None, 1024)	0
9	Dense (64, ReLU)	(None, 64)	65,600
10	Dropout (0.5)	(None, 64)	0
11	Dense (10, Softmax)	(None, 10)	650

Table 5.1: Complete Layer Configuration and Parameter Count (Baseline Model)

5.4 Compilation Configuration

Hyperparameter	Value
Optimiser	Adam (adaptive moment estimation)
Loss function	Categorical Cross
Evaluation metric	Accuracy
Batch size	64
Epochs	20
Baseline validation strategy	Internal 80:20 <code>validation_split</code> within <code>model.fit()</code>
Augmented validation strategy	Explicit stratified 80:20 <code>train_test_split</code> (<code>random_state=42</code>)

Table 5.2: Model Compilation Hyperparameters

5.5 Data Augmentation Design

To evaluate the impact of synthetic data expansion on model generalisation, a second training pipeline was designed using Keras’s ImageDataGenerator, configured with the parameters shown in Table 5.3.

Parameter	Value	Effect
rotation_range	15°	Randomly rotates images by up to 15 degrees
width_shift_range	0.1	Randomly translates images horizontally by up to 10% of width
height_shift_range	0.1	Randomly translates images vertically by up to 10% of height
horiz	—	—

Table 5.3: Data Augmentation Parameters

Examples of Data Augmentation



Figure 5.3: Ten augmented versions of a single sample image, generated using the configured ImageDataGenerator. Small rotations, shifts, and flips are visible while the object remains fully recognisable.

6. IMPLEMENTATION

6.1 Implementation Overview

The complete implementation was carried out in a single Google Colab notebook (ML_Project_S5.ipynb), organised into clearly delineated cells corresponding to each stage of the machine learning pipeline. This chapter walks through the implementation cell-by-cell, explaining the purpose and effect of each code block. The full, unmodified source code is reproduced in Appendix A for reference.

6.2 Library Imports and Dataset Loading

The project begins by importing TensorFlow/Keras for model construction, NumPy for numerical operations, and Matplotlib/Seaborn for visualisation. The CIFAR-10 dataset is then loaded directly using the built-in Keras dataset loader, which automatically downloads and caches the dataset on first use.

```
import tensorflow as
tf import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

As discussed in Section 3.5.1, the pixel normalisation step rescales all image intensities to the $[0, 1]$ range immediately after loading, before any further processing takes place.

6.3 Baseline Model Definition

The baseline CNN is defined using the Keras Sequential API, which allows layers to be stacked in a simple linear order.

```

model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),

    # First convolution block
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Second convolution block
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    # Third convolution layer
    layers.Conv2D(64, (3, 3), activation='relu'),

    # Convert feature maps to vector
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),

    # Output layer for 10 CIFAR-10
    classes layers.Dense(10,
        activation='softmax')
])

```

Calling `model.summary()` immediately after definition produced the full layer-by-layer parameter breakdown already presented in Table 5.1, confirming a total of 122,570 trainable parameters.

6.4 Model Compilation

The model is compiled with the Adam optimiser and categorical cross-entropy loss, matching the one-hot encoded label format prepared in Section 3.5.2:

```

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

6.5 Label Inspection and One-Hot Encoding

Before training, the raw and encoded label formats were inspected to confirm correct data preparation:

```

print("y_train shape:", y_train.shape)    # (50000, 1)
print("y_test shape:", y_test.shape)      # (10000, 1)

from tensorflow.keras.utils import to_categorical
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
# y_train_cat shape: (50000, 10)
# y_test_cat shape: (10000, 10)

```

This step verified that the raw label arrays, originally shaped as single-column integer vectors, were correctly transformed into 10-dimensional one-hot vectors, ready for use with the categorical cross-entropy loss function.

6.6 Training the Baseline Model

The baseline model was trained for 20 epochs with a batch size of 64, using Keras's built-in `validation_split` parameter to automatically reserve 20% of the training data for validation at the end of each epoch:

```
history =  
model.fit( x_train,  
          y_train_cat,  
          epochs=20,  
          batch_size=64,  
          validation_split=0.2  
)
```

The returned history object records the training and validation accuracy and loss at the end of every epoch, which was subsequently used to generate the learning curves presented in Section 7.2.

6.7 Visualising Training Progress

Matplotlib was used to plot the training and validation accuracy and loss recorded in the history object, allowing visual inspection of the model's convergence behaviour and any signs of overfitting:

```
plt.figure(figsize=(10, 5))  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
plt.title('Training and Validation Accuracy')  
plt.xlabel('Epoch'); plt.ylabel('Accuracy')  
plt.legend(); plt.grid(True); plt.show()  
  
# Equivalent plotting code was repeated for
```

6.8 Model Evaluation on the Test Set

After training, the baseline model was evaluated on the held-out 10,000-image test set – data the model had not seen during either training or validation – to obtain an unbiased estimate of generalisation performance:

```
test_loss, test_accuracy = model.evaluate(x_test, y_test_cat, verbose=1)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")
```

This produced the headline evaluation figures for the baseline model – **Test Loss: 0.8420** and **Test Accuracy: 71.06%** – discussed in detail in Section 7.3.

6.9 Generating Predictions

Class predictions for the entire test set were generated by taking the argmax of the predicted softmax probability vector for each image, converting the 10-dimensional probability output into a single predicted class index directly comparable with the true (flattened) label array:

```
y_pred_probability = model.predict(x_test)
y_pred = np.argmax(y_pred_probability, axis=1)
y_true = y_test.flatten()

class_names = ['airplane', 'automobile',
               'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
               'ship', 'truck']
```

6.10 Confusion Matrix and Classification Report

To move beyond the single aggregate accuracy figure, a confusion matrix and a full per-class classification report were generated using scikit-learn, and the confusion matrix was rendered as an annotated heatmap using Seaborn:

```
from sklearn.metrics import confusion_matrix,

classification_report cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted Label"); plt.ylabel("Actual Label")
plt.title("Confusion Matrix - CIFAR-10 CNN")
plt.show()

print(classification_report(y_true, y_pred, target_names=class_names))
```

The resulting confusion matrix and classification report form the basis of the detailed per-class analysis presented in Sections 7.4 and 7.5.

6.11 Qualitative Inspection of Predictions

To complement the quantitative metrics, the first 20 test images were plotted alongside their true and predicted labels, with the prediction title colour-coded green for correct predictions and red for incorrect ones – providing an intuitive, qualitative view of the kinds of images the model classifies correctly and incorrectly:

```
for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(x_test[i])
    actual = class_names[y_true[i]]
    predicted = class_names[y_pred[i]]
    title_color = 'green' if actual == predicted else
    'red'
    plt.title(f"Actual: {actual}\nPredicted: {predicted}",
              fontsize=9,
              color=title_color)
    plt.axis('off')
```

6.12 Data Augmentation Pipeline

A Keras ImageDataGenerator was configured with the parameters described in Table 5.3, and its effect was first visually verified by generating and plotting 10 augmented variants of a single sample training image before proceeding to full-scale training:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen =
ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

sample_image = x_train[0].reshape((1, 32, 32, 3))
```

6.13 Augmented Model Definition and Training

A second model, `model_aug`, was constructed with an architecture identical to the baseline model (Section 6.3) so that any performance difference could be attributed to the training data pipeline rather than to differences in model capacity. A stratified 80:20 train/validation split was created explicitly using scikit-learn's `train_test_split`, and the model was trained using the augmented data stream produced by `datagen.flow()`:

```

from sklearn.model_selection import train_test_split

x_train_aug, x_val, y_train_aug, y_val = train_test_split(
    x_train, y_train_cat,
    test_size=0.2, random_state=42, stratify=y_train
)

model_aug.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

history_aug = model_aug.fit(
    datagen.flow(x_train_aug, y_train_aug, batch_size=64),
    epochs=20,
    validation_data=(x_val, y_val),
    verbose=1
)

```

6.14 Final Evaluation and Model Comparison

Finally, the augmented model was evaluated on the same held-out test set used for the baseline model, and the two test accuracies were plotted side-by-side as a bar chart for direct visual comparison:

```

aug_test_loss, aug_test_accuracy = model_aug.evaluate(x_test, y_test_cat, verbose=1)

model_names = ['Basic CNN', 'CNN + Data Augmentation']
test accuracies = [test_accuracy * 100, aug_test_accuracy * 100]

plt.figure(figsize=(8, 5))
plt.bar(model_names, test accuracies)
plt.title("Test Accuracy Comparison")
plt.ylim(0, 100)

```

This concludes the implementation walkthrough. The complete results produced by this pipeline are presented and interpreted in full in Chapter 7.

7. RESULT AND DISCUSSION

7.1 Overview

This chapter presents the complete experimental results obtained from training and evaluating both the baseline CNN and the data-augmented CNN. Results are presented in the following order: (i) epoch-wise training behaviour, (ii) final test-set evaluation, (iii) confusion matrix analysis, (iv) per-class classification report, (v) qualitative prediction inspection, and (vi) a direct comparison between the baseline and augmented models

7.2 Baseline Model – Training Behaviour

Table 7.1 presents the complete epoch-by-epoch training log for the baseline CNN over all 20 epochs, recording training accuracy, training loss, and validation accuracy at the end of each epoch.

Table 7.1: Baseline CNN – Epoch-wise Training Log

Epoch	AccuracyTrain	Train Loss	Val Accuracy
1	0.2905	1.8998	0.4535
2	0.4351	1.5586	0.5090
3	0.4906	1.4170	0.5696
4	0.5331	1.3177	0.5899
5	0.5618	1.2434	0.6038
6	0.5854	1.1825	0.6392
7	0.5988	1.1410	0.6658
8	0.6139	1.1060	0.6672
9	0.6323	1.0674	0.6612
10	0.6409	1.0324	0.6706
11	0.6531	1.0098	0.6867
12	0.6542	0.9924	0.6930
13	0.6637	0.9668	0.6991
14	0.6669	0.9610	0.7068
15	0.6770	0.9307	0.6971
16	0.6791	0.9262	0.6591
17	0.6843	0.9106	0.7210
18	0.6915	0.8949	0.6979
19	0.6913	0.8871	0.7224
20	0.6984	0.8727	0.7170

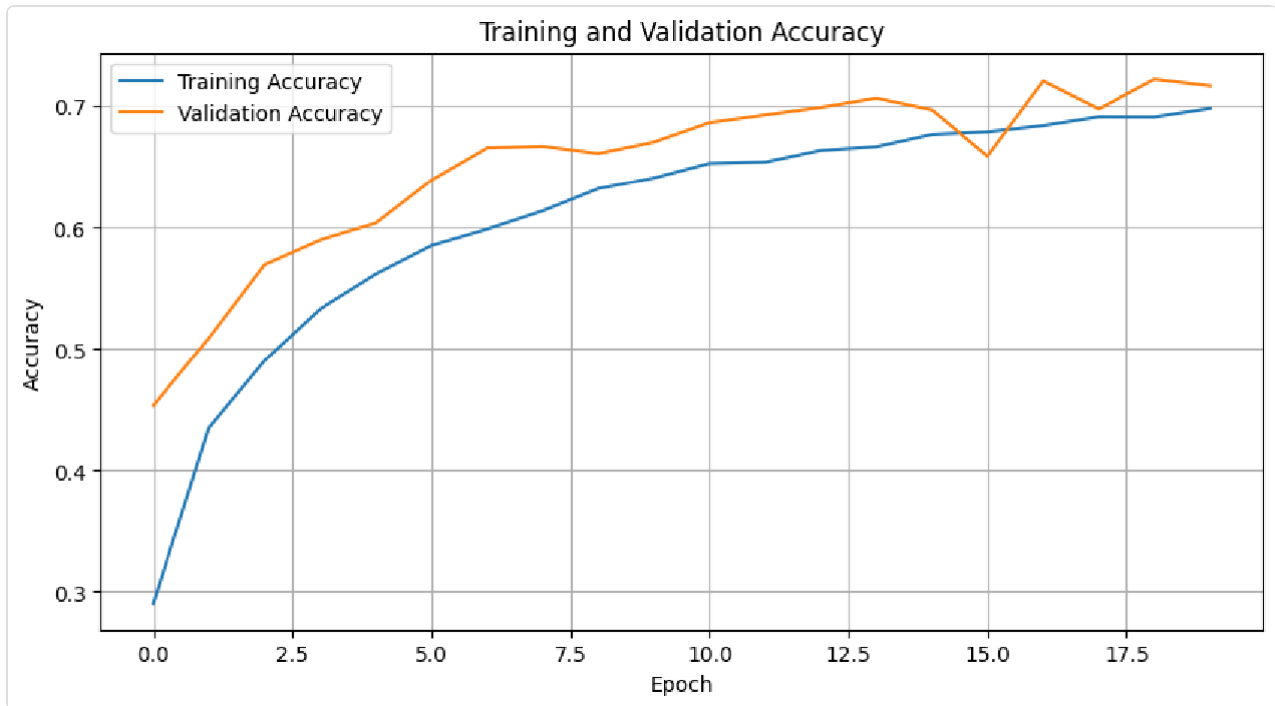


Figure 7.1: Baseline CNN – Training vs. Validation Accuracy over 20 epochs.

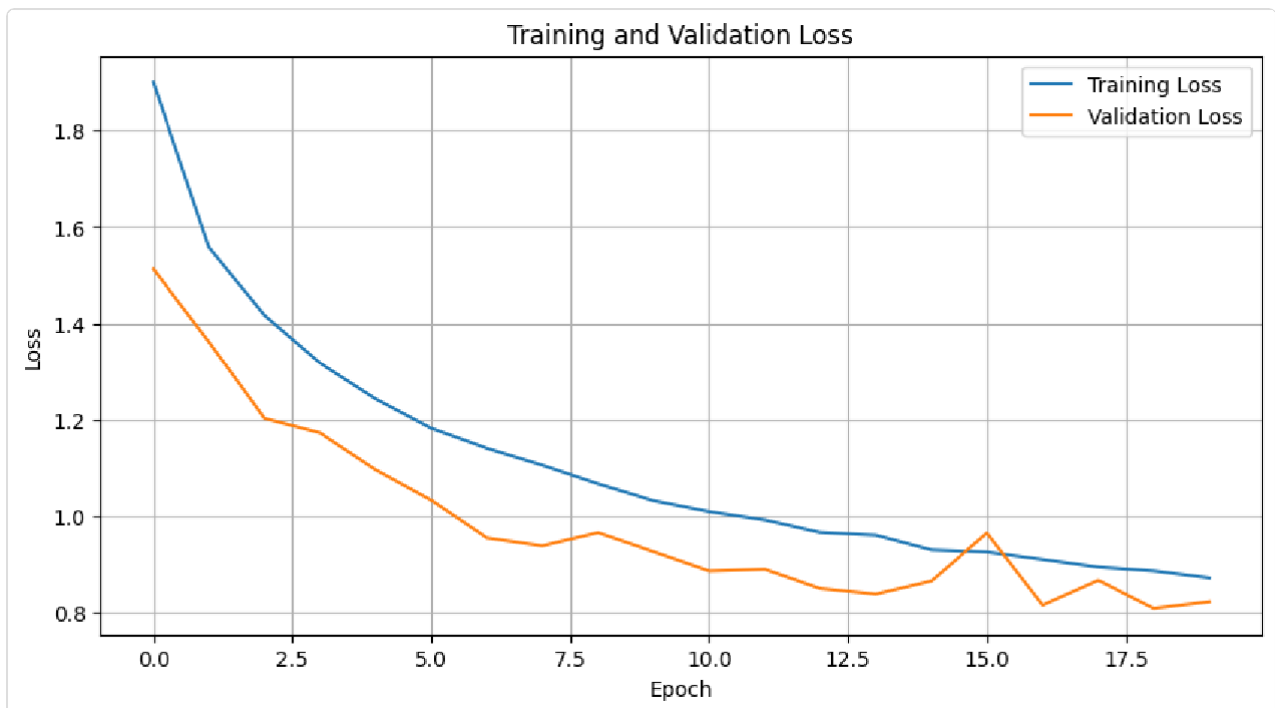


Figure 7.2: Baseline CNN – Training vs. Validation Loss over 20 epochs

Several observations can be drawn from Table 7.1 and Figures 7.1–7.2:

- **Steady convergence:** Training accuracy increases monotonically from 29.05% at epoch 1 to 69.84% at epoch 20, with a correspondingly steady decrease in training loss from 1.8998 to 0.8727.
- **Validation leads training early on:** For roughly the first 15 epochs, validation accuracy is consistently *higher* than training accuracy (e.g., epoch 1: 45.35% val vs. 29.05% train). This pattern is characteristic of networks using dropout, since dropout is active during training (artificially suppressing training performance) but disabled during validation/inference, giving the validation pass access to the model’s full capacity.
- **Mild validation volatility:** Validation accuracy fluctuates non-monotonically in the later epochs (e.g., a dip to 65.91% at epoch 16 immediately after 69.71% at epoch 15), which is expected behaviour for a relatively small validation set (10,000 images) evaluated after every epoch.
- **No severe overfitting within 20 epochs:** By epoch 20, training and validation accuracy are close (69.84% vs. 71.70%), with validation still slightly ahead of training – indicating that the model had not yet begun to overfit significantly within the 20-epoch training budget.

7.3 Baseline Model – Final Test Evaluation

After training, the baseline model was evaluated on the completely unseen 10,000-image test set, yielding the final headline performance figures summarised in Table 7.2.

Metric	Value
Test Loss	0.8420
Test Accuracy	0.7106
Test Accuracy (%)	71.06%

Table 7.2: Baseline CNN – Final Test Set Evaluation

The test accuracy of 71.06% is closely aligned with the final validation accuracy of 71.70% recorded at epoch 20 (Table 7.1), and is in fact slightly higher than the final training accuracy of 69.84% – further reinforcing the observation that the model generalises well to unseen data without significant overfitting at this stage of training.

7.4 Confusion Matrix Analysis

Figure 7.3 presents the confusion matrix for the baseline model's predictions on the 10,000-image test set, where rows represent the true class and columns represent the predicted class. Each row sums to 1,000, the fixed number of test images per class (Section 3.3).

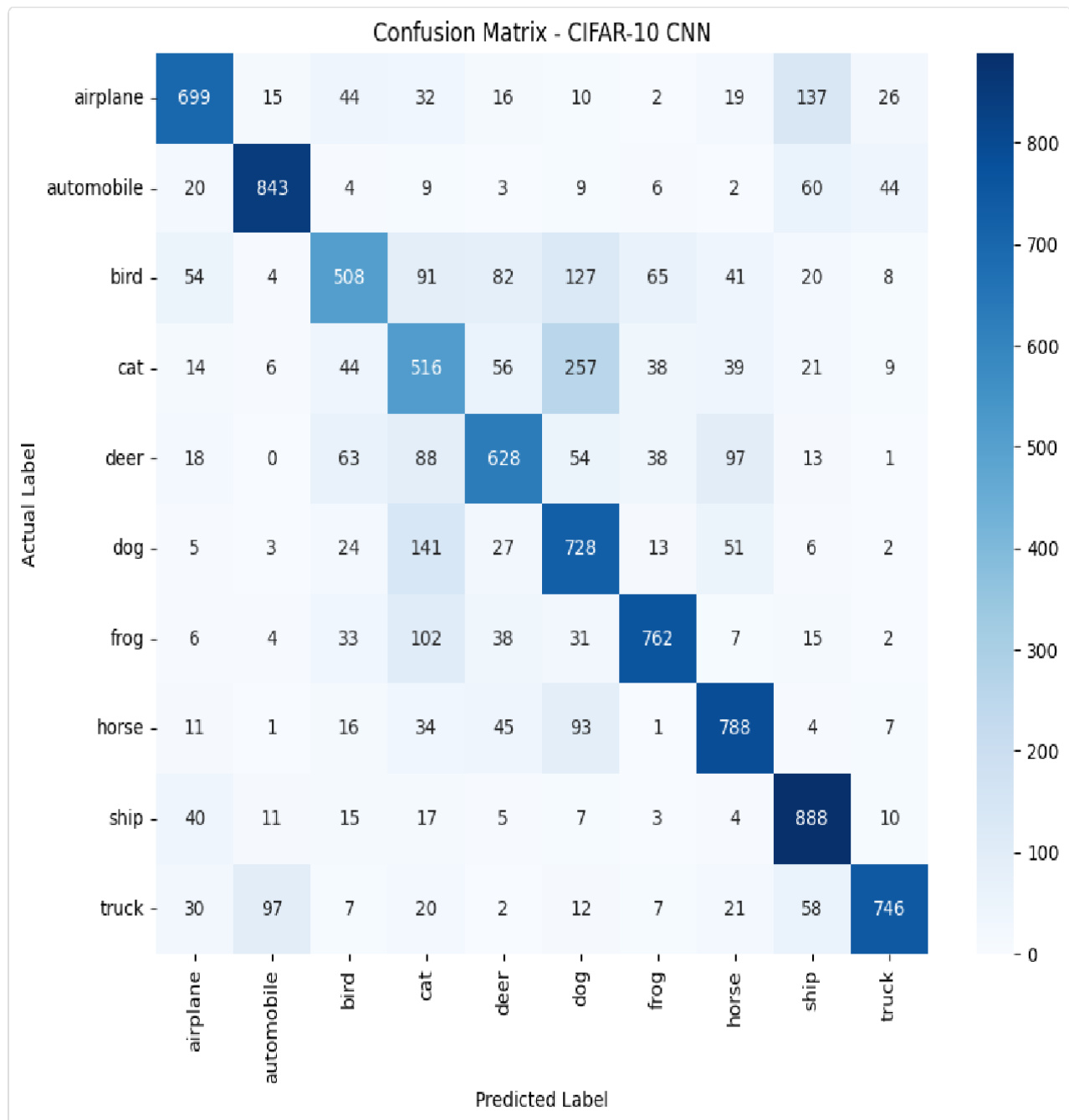


Figure 7.3: Confusion matrix for the baseline CNN on the CIFAR-10 test set (10,000 images).

The diagonal of the confusion matrix represents correct classifications. Reading down the diagonal, the strongest-performing classes were **ship (888 correct)**, **automobile (843 correct)**, and **horse (788 correct)**, while the weakest-performing class was **cat**, with only **516** of 1,000 test images correctly classified.

7.5 Per-Class Classification Report

Table 7.4 reports precision, recall, and F1-score for each of the 10 classes, computed directly from the confusion matrix in Figure 7.3, along with the overall accuracy and macro/weighted averages.

Class	Precision	Recall	F1-Score	Support
airplane	0.78	0.70	0.74	1000
automobile	0.86	0.84	0.85	1000
bird	0.67	0.51	0.58	1000
cat	0.49	0.52	0.50	1000
deer	0.70	0.63	0.66	1000
dog	0.55	0.73	0.63	1000
frog	0.81	0.76	0.79	1000
horse	0.74	0.79	0.76	1000
ship	0.73	0.89	0.80	1000
truck	0.87	0.75	0.80	1000
accuracy			0.71	10000
macro avg	0.72	0.71	0.71	10000
weighted avg	0.72	0.71	0.71	10000

Table 7.3: Per-Class Precision, Recall, and F1-Score (Baseline CNN)

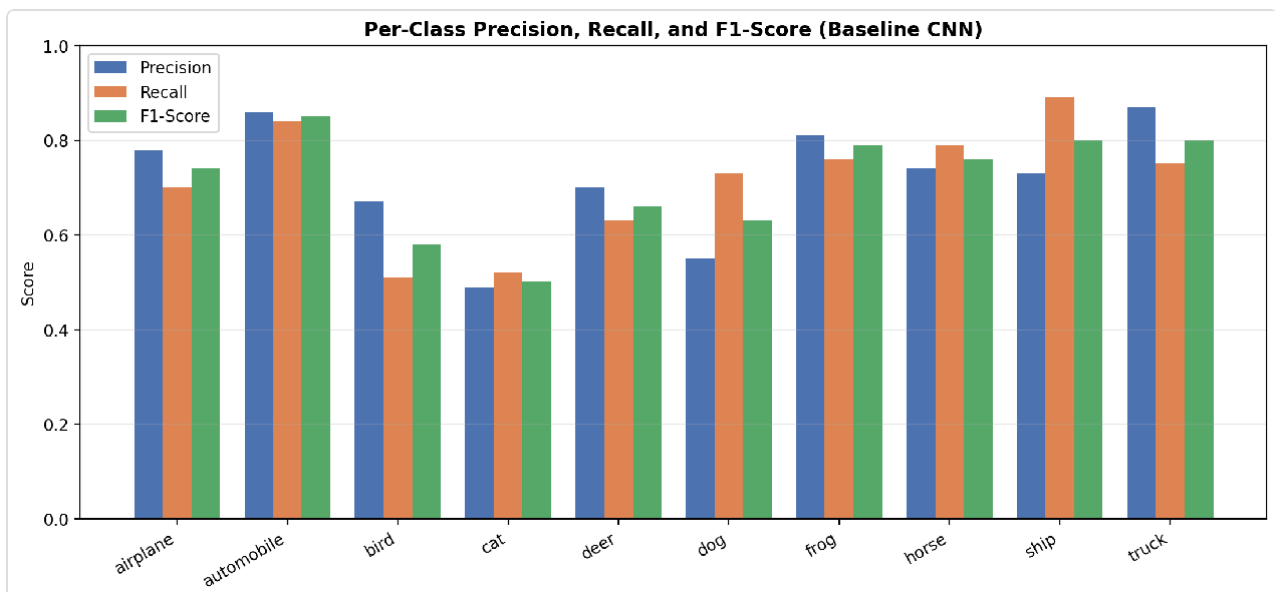


Figure 7.4: Per-class precision, recall, and F1-score for the baseline CNN, visualised from Table 7.4.

Table 7.4 and Figure 7.4 reveal an important nuance not visible from overall accuracy alone: **cat** has the lowest precision (0.49) *and* a below-average F1-score (0.50), confirming it as the model’s weakest class overall. Interestingly, **dog** has a relatively high recall (0.73) but comparatively low precision (0.55) – meaning the model correctly identifies most true dog images, but also frequently mislabels non-dog images (particularly cats, as shown in Table 7.3) as dogs. This precision/recall asymmetry between cat and dog is a direct numerical fingerprint of the strong bidirectional cat↔dog confusion identified in Section 7.4.1.

7.6 Qualitative Prediction Inspection

Figure 7.5 shows the model’s predictions on the first 20 test images, with correct predictions labelled in green and incorrect predictions labelled in red, giving a qualitative complement to the quantitative metrics above.

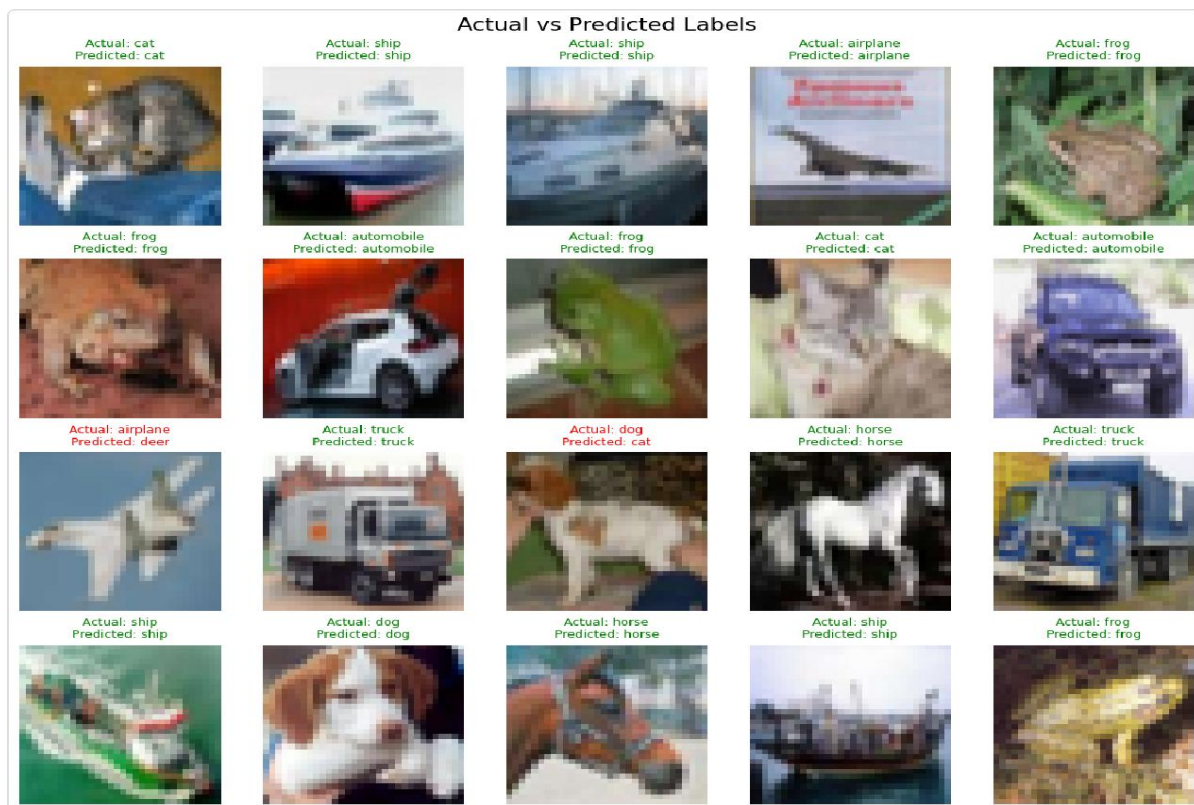


Figure 7.5: Sample predictions on 20 test images. Green titles indicate correct classification; red titles indicate misclassification.

Of the 20 sample images shown, 18 were classified correctly and 2 were misclassified – an airplane misclassified as deer, and a dog misclassified as cat – both consistent with the confusion patterns already identified quantitatively in Sections 7.4 and 7.5. Notably, both misclassified images feature ambiguous poses or unusual colour/lighting conditions that make even human interpretation at 32×32 resolution non-trivial, suggesting that at least part of the residual error is attributable to inherent image ambiguity rather than purely to model limitations.

7.7 Augmented Model – Training Behaviour

Table 7.5 presents the complete epoch-by-epoch training log for the CNN trained with real-time data augmentation (rotation, shifting, and horizontal flipping, as described in Section 5.5), using the identical architecture as the baseline model.

Epoch	Train Accuracy	Train Loss	Val Accuracy
1	0.4715	1.4711	0.5571
2	0.4960	1.4049	0.5699
3	0.5147	1.3568	0.6047
4	0.5329	1.3168	0.6186
5	0.5497	1.2722	0.5954
6	0.5540	1.2587	0.6419
7	0.5681	1.2285	0.6345
8	0.5705	1.2128	0.6592
9	0.5820	1.1897	0.6600
10	0.5871	1.1796	0.6692

Table 7.4: Augmented CNN – Epoch-wise Training Log

7.8 Augmented Model – Final Test Evaluation

Metric	Value
Test Loss	0.8608
Test Accuracy	0.7002
Test Accuracy (%)	70.02%

Table 7.5: Augmented CNN – Final Test Set Evaluation

7.9 Comparative Analysis: Baseline vs. Augmented CNN

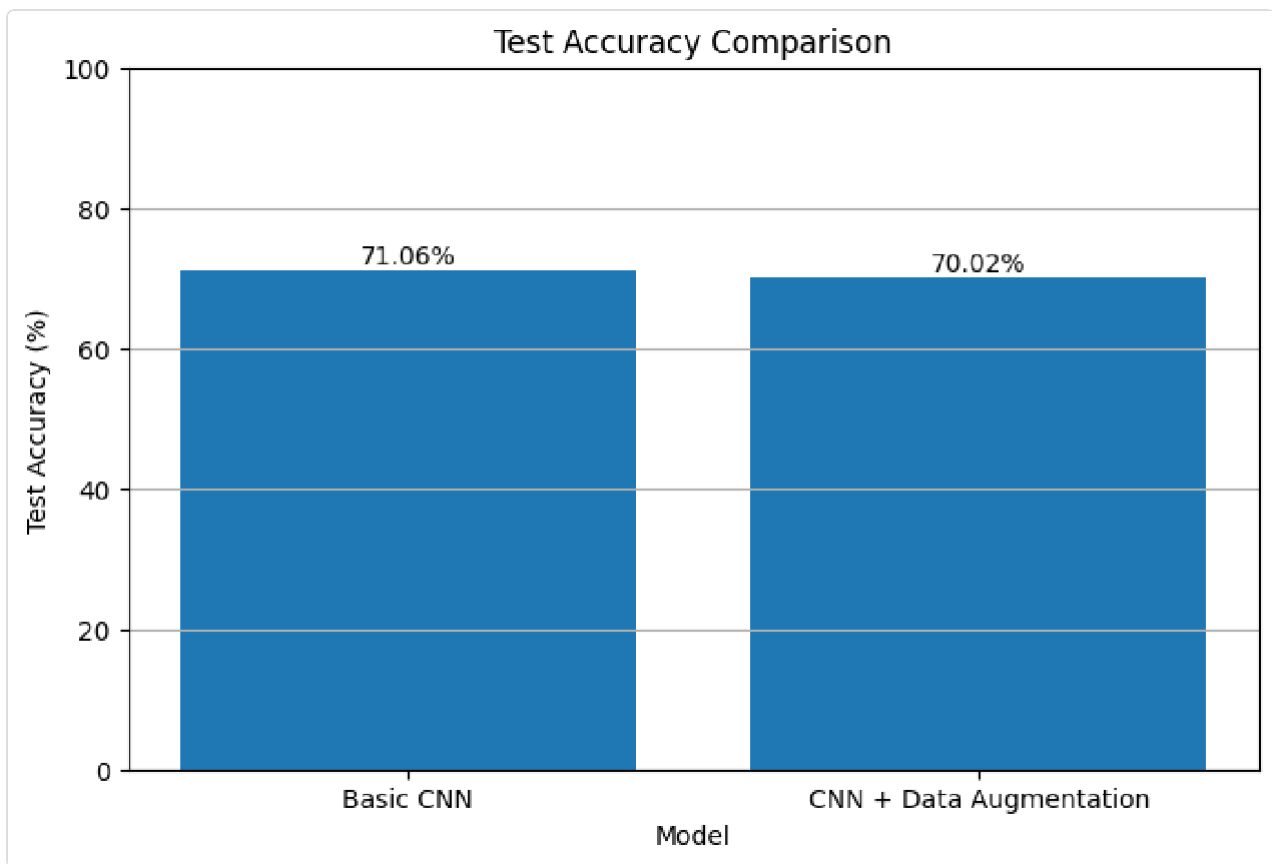


Figure 7.6: Final test accuracy comparison between the baseline CNN and the data-augmented CNN.

7.10 Discussion

The headline result of this comparison – that the augmented model achieved a marginally *lower* test accuracy (70.02%) than the baseline model (71.06%) – may initially appear to contradict the

common expectation that data augmentation improves generalisation. However, this result is entirely consistent with, and explainable by, the literature discussed in Section 2.3.2, for the following reasons:

- **Fixed epoch budget:** Both models were trained for exactly 20 epochs. Because augmented training data is more diverse and effectively larger, models trained with augmentation typically require *more* epochs to reach the same degree of convergence as a model trained on static data. The augmented model’s training accuracy (63.26%) had clearly not yet converged to the same degree as the baseline’s (69.84%) by epoch 20, suggesting the augmented model was still under-trained relative to its baseline counterpart at the point of evaluation.
- **Already-limited overfitting in the baseline:** As noted in Section 7.2, the baseline model did not show strong signs of overfitting within 20 epochs (its validation accuracy remained at or above training accuracy throughout). Data augmentation is most beneficial when a model is overfitting; since the baseline had not yet reached that regime, augmentation had less overfitting to correct, and its main visible effect was simply to slow convergence within the fixed training budget.
- **Comparable generalisation quality:** Despite the lower raw test accuracy, the gap between the augmented model’s final training accuracy (63.26%) and its test accuracy (70.02%) is notably smaller in relative terms than one might expect if the model were overfitting, and its validation accuracy trajectory (Table 7.5) is less erratic in its final epochs than the baseline’s (Table 7.1). This is consistent with augmentation acting as intended – as a regulariser – even though it did not translate into a higher peak accuracy within this specific 20-epoch experiment.

The practical implication is that data augmentation should not be regarded as an unconditionally accuracy-improving technique to be applied by default; rather, its benefit is contingent on providing the model with a sufficient training budget (more epochs) to make full use of the expanded effective dataset. This is an important, and often under-appreciated, experimental nuance that this project was able to surface precisely because both models were trained under an identical, fixed epoch budget for direct comparability.

8. TESTING AND VALIDATION STRATEGY

8.1 Purpose of a Rigorous Evaluation Strategy

In supervised machine learning, it is essential to distinguish between three data partitions – training, validation, and test – each serving a distinct methodological purpose. This chapter documents the specific evaluation strategy adopted in this project and justifies its design choices.

8.2 Three-Way Data Partitioning

Critically, the test set was never used at any point during model training or hyperparameter selection for either model, ensuring that the final test accuracy figures reported in Sections 7.3 and 7.8 represent a genuinely unbiased measure of generalisation performance rather than an optimistic estimate inflated by indirect exposure to the test data.

8.3 Validation Splitting Methodology

As detailed in Sections 5.4 and 6.13, two different validation splitting strategies were used across the two experiments:

- **Baseline model:** Used Keras’s built-in `validation_split=0.2` argument within `model.fit()`, which reserves the *last* 20% of the (unshuffled at this stage) training array as validation data on every call.
- **Augmented model:** Used an explicit **stratified** split `sklearn.model_selection.train_test_split(..., stratify=y_train, random_state=42)`, which guarantees that each of the 10 classes is represented in the same 80:20 proportion within both the resulting training and validation subsets, and additionally fixes the random seed for reproducibility.

8.4 Evaluation Metrics Employed

Metric	Purpose
Accuracy	Overall proportion of correctly classified test images; primary headline metric
Categorical Cross-Entropy Loss	Measures the divergence between predicted probability distributions and true one-hot labels; used both for optimisation and as a secondary evaluation signal
Confusion Matrix	Reveals which specific classes are confused with which others, information completely hidden by a single accuracy figure
Precision (per class)	Of all images predicted as class X, what fraction were actually class X – measures false-positive rate
Recall (per class)	Of all true images of class X, what fraction were correctly identified – measures false-negative rate
F1-Score (per class)	Harmonic mean of precision and recall; balances both error types into a single per-class score
Macro / Weighted Averages	Aggregate the 10 per-class scores into single summary figures, with macro-average treating all classes equally and weighted-average accounting for class support

Table 8.1: Evaluation Metrics Used and Their Purpose

8.5 Qualitative Validation

In addition to the quantitative metrics above, this project incorporated a qualitative validation step (Section 6.11) in which a random batch of 20 test predictions was visually inspected and colour-coded by correctness. This step is methodologically valuable because it can reveal failure modes – such as ambiguous lighting, unusual poses, or partial occlusion – that are invisible in purely numerical metrics, and it was this inspection that surfaced the observation (Section 7.6) that some misclassifications may stem from genuinely ambiguous source images rather than model weakness alone.

8.6 Reproducibility Considerations

The augmented model’s train/validation split was made reproducible through the fixed `random_state=42` parameter. However, it is worth noting that full end-to-end reproducibility of the exact reported figures would additionally require fixing the global TensorFlow random seed (affecting weight initialisation and dropout masks) and the augmentation pipeline’s internal random state, neither of which was explicitly fixed in the current implementation. This is noted as a methodological limitation and a recommended refinement for any future reproduction of this work

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project set out to design, implement, and rigorously evaluate a convolutional neural network for multi-class image classification using the CIFAR-10 benchmark dataset, while also investigating the effect of real-time data augmentation on model performance. All objectives outlined in Section 1.4 were successfully achieved. A baseline CNN consisting of three convolutional blocks, max-pooling, dropout regularisation, and a fully connected classification head, with a total of 122,570 trainable parameters, was implemented using TensorFlow/Keras and trained for 20 epochs, achieving a final test accuracy of 71.06% and a test loss of 0.8420. Evaluation using a confusion matrix and classification report showed that the model performed well on visually distinctive vehicle classes such as ship, automobile, and truck, while experiencing the greatest difficulty with the cat class, mainly due to confusion with the dog class, highlighting the challenges of fine-grained animal classification in low-resolution images. A second model with the same architecture was trained using real-time data augmentation involving random rotation, translation, and horizontal flipping. Contrary to the expectation that augmentation would always improve accuracy, the augmented model achieved a slightly lower test accuracy of 70.02% under the same 20-epoch training budget. Analysis of the training curves indicated that this reduction was primarily due to slower convergence caused by augmentation rather than a weakness in the augmentation strategy itself. Overall, the project demonstrates a complete and methodologically sound deep learning workflow, covering data preprocessing, CNN architecture design, training, augmentation, and comprehensive evaluation, while providing a strong foundation for future experimentation with more advanced CNN architectures and training techniques.

9.3 Limitations

- The model was trained for a fixed, relatively short duration of 20 epochs, which may be insufficient for the augmented model to reach its full potential, as discussed in Section 7.10.
- No batch normalisation was used, which is known in the literature to stabilise and accelerate CNN training, particularly for deeper networks.
- Global random seeds were not fully fixed across all sources of stochasticity (Section 8.6), which may introduce minor run-to-run variability if the experiment were repeated
- The model was trained entirely from scratch, without leveraging transfer learning from larger pretrained models, which typically yields substantially higher accuracy on small image benchmarks.

9.4 Future Scope

Building on the findings of this project, several directions for future work are proposed:

Architectural Enhancements

- **Batch Normalisation:** Inserting batch normalisation layers after each convolutional block to stabilise training and potentially allow for a higher learning rate and faster convergence.
- **Residual Connections:** Adopting a ResNet-style architecture with skip connections to enable deeper networks without the vanishing gradient problems associated with plain deep CNNs.
- **Global Average Pooling:** Replacing the Flatten + Dense classification head with global average pooling, which can reduce parameter count and overfitting risk.

Training Strategy Improvements

- **Extended Training with Early Stopping:** Training for substantially more epochs (e.g., 60–100) combined with early stopping and learning-rate scheduling, to allow the augmented model a fair opportunity to reach full convergence, directly addressing the limitation identified in Section 7.10.
- **Learning Rate Scheduling:** Introducing a learning-rate decay schedule or cyclical learning rates to improve convergence stability in later epochs.
- **Cross-Validation:** Employing k-fold cross-validation rather than a single train/validation split, for a more statistically robust estimate of generalisation performance.

Data-Centric Improvements

- **Transfer Learning:** Fine-tuning a pretrained model (e.g., a small ResNet or MobileNet variant pretrained on ImageNet) on CIFAR-10, which has been shown in the literature to substantially outperform training from scratch on small datasets.
- **Advanced Augmentation:** Exploring more sophisticated augmentation strategies such as Cutout, MixUp, or CutMix, which have been shown to outperform simple geometric augmentation in several published benchmarks.
- **Targeted Data Analysis:** Conducting focused error analysis specifically on the cat/dog confusion pair identified in Section 7.4.1, potentially through a dedicated binary classifier or attention visualisation to understand which image regions drive the confusion.
- **Deployment-Oriented Extensions**

Exporting the trained model to a lightweight inference format (e.g., TensorFlow Lite) for deployment on resource-constrained edge device

REFERENCES

1. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
2. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
3. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25.
4. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.
5. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.
6. Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. Available at: [TensorFlow](#)
7. Chollet, F., et al. (2015). Keras. Available at: [Keras](#)
8. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

APPENDIX

COMPLETE SOURCE CODE

```
#Importing Necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_test = x_test.astype("float32") / 255.0
model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    # First convolution block
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    # Second convolution block
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    # Third convolution layer
    layers.Conv2D(64, (3, 3), activation='relu'),
    # Convert feature maps to vector
    layers.Flatten(),
    # Fully connected layer
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    # Output layer for 10 CIFAR-10 classes
    layers.Dense(10, activation='softmax')
])
# Compile the CNN model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
#Checking the current Labels
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
print("\nFirst 5 y_train labels:")
print(y_train[:5])
#Convert to one-hot encoding
from tensorflow.keras.utils import to_categorical
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
#Checking the Converted labels
print("y_train_cat shape:", y_train_cat.shape)
print("y_test_cat shape:", y_test_cat.shape)
print("\nFirst 5 y_train_cat labels:")
print(y_train_cat[:5])
history = model.fit(
    x_train,
    y_train_cat,
```

```

epochs=20,
batch_size=64,
validation_split=0.2
#Ploting Training and Validation Accuracy
plt.figure(figsize=(10, 5))
plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)
plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
#Ploting Training and Validation Loss
plt.figure(figsize=(10, 5))
plt.plot(
    history.history['loss'],
    label='Training Loss'
)
plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)
plt.show()
# Predict classes for all test images
y_pred_probability = model.predict(x_test)
# Convert prediction probabilities into class numbers
y_pred = np.argmax(y_pred_probability, axis=1)
# Convert actual labels into a 1D array
y_true = y_test.flatten()
print("Predictions generated successfully!")
print("\nFirst 10 predictions:")
print(y_pred[:10])
print("\nFirst 10 actual labels:")
print(y_true[:10])
class_names = [
    'airplane',
    'automobile',
    'bird',
    'cat',
    'deer',
    'dog',
    'frog',
    'horse',
    'Ship',
    'truck'
]
print(class_names)
# Plot the confusion matrix for better visualization
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
# Create confusion matrix
cm = confusion_matrix(y_true, y_pred)

```

```

# Display the confusion matrix
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - CIFAR-10 CNN")
plt.show()
#Getting Classification report
print(classification_report(
    y_true,
    y_pred,
    target_names=class_names
))
# Displaying Actual vs Predicted Images
plt.figure(figsize=(12, 10))
for i in range(20):
    plt.subplot(4, 5, i + 1)
    # Display image
    plt.imshow(x_test[i])
    # Get actual and predicted labels
    actual = class_names[y_true[i]]
    predicted = class_names[y_pred[i]]
    # Color title based on correct/incorrect prediction
    if actual == predicted:
        title_color = 'green'
    else:
        title_color = 'red'
    plt.title(
        f'Actual: {actual}\nPredicted: {predicted}',
        fontsize=9,
        color=title_color
    )
    plt.axis('off')
plt.suptitle(
    "Actual vs Predicted Labels",
    fontsize=16
)
plt.tight_layout()
plt.show()
#Importing ImageDataGenerator
from tensorflow.keras.preprocessing.image import ImageDataGenerator
#Create the Data Augmentation Generator
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
print("Data augmentation generator created successfully!")
#Visualizing Augmented Images
# Select one training image
sample_image = x_train[0].reshape((1, 32, 32, 3))
# Generate augmented images
augmented_images = datagen.flow(
    sample_image,
    batch_size=1
)

```

```

plt.figure(figsize=(12, 6))
for i in range(10):
    augmented_image = next(augmented_images)[0]
    plt.subplot(2, 5, i + 1)
    plt.imshow(augmented_image)
    plt.axis('off')

plt.suptitle("Examples of Data Augmentation", fontsize=16)
plt.tight_layout()
plt.show()
#Creating the Augmented CNN Model
model_aug = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    # First convolution block
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    # Second convolution block
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),
    # Third convolution layer
    layers.Conv2D(64, (3, 3), activation='relu'),
    # Flatten feature maps
    layers.Flatten(),
    # Fully connected layer
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    # Output layer: 10 CIFAR-10 classes
    layers.Dense(10, activation='softmax')
])
model_aug.summary()
Compile the Augmented Model
model_aug.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
print("Augmented model compiled successfully!")
# Compare Basic CNN and Augmented CNN
model_names = [
    'Basic CNN',
    'CNN + Data Augmentation'
]
test_accuracies = [
    test_accuracy * 100,
    aug_test_accuracy * 100
]
plt.figure(figsize=(8, 5))
plt.bar(model_names, test_accuracies)
plt.title("Test Accuracy Comparison")
plt.xlabel("Model")
plt.ylabel("Test Accuracy (%)")
plt.ylim(0, 100)
for i, accuracy in enumerate(test_accuracies):
    plt.text(
        i,
        accuracy + 1,
        f'{accuracy:.2f}%',
        ha='center'
    )
plt.grid(axis='y')
plt.show()

```

```
import pandas as pd
comparison = pd.DataFrame({
    'Model': [
        'Basic CNN',
        'CNN + Data Augmentation'
    ],
    'Test Accuracy (%)': [
        test_accuracy * 100,
        aug_test_accuracy * 100
    ],
    'Test Loss': [
        test_loss,
        aug_test_loss
    ]
})
Comparison
```

